Qualifying Dissertation for the Transfer of Status

Theo Wang

November 12, 2026

1 Introduction

2 Literature Review

2.1 Foundations of Probabilistic Programming

From Bayesian Networks to Probabilistic Programming A central tool in Bayesian probabilistic modelling is Bayesian networks [TODO: cite], a graphical representation of a joint distribution. Formally, a Bayesian network over a certain set of variables V is a directed acyclic graph over V, a domain D_x for each variable $x \in V$, and for each variable $v \in V$, a conditional distribution $P(x \mid \mathsf{Parents}(x))$. This data can be seen as a Markov factorisation of a joint distribution

$$P(x_1 ... x_n) = \prod_{x \in V} P(x \mid \mathsf{Parents}(x))$$

Bayesian networks are useful because their graphical structure encodes the conditional independencies between variables (a variable is assumed to only depend on its parents), exploited to make Bayesian inference (computing conditional distributions) more efficient. [TODO: cite again]

For instance consider the scenario where we are trying to model a car, which has fuel with probability 0.8, battery with probability 0.9, and only starts if it has both fuel and battery. Then we can model the joint distribution between fuel, $\mathsf{battery}$ and $\mathsf{carStarts}$ with the Bayesian network in fig. 1.

Probabilistic Programming languages can be seen as a compositional generalisation of Bayesian networks: they provide a compositional syntax for specifying probabilistic models and performing inference on them. Our running example, for instance, can be represented by the program in fig. 2. Like for Bayesian networks, probabilistic programs represent joint distributions (lines 1-3 of fig. 2 represent the joint distribution represented by the Bayesian network). Unlike Bayesian networks, we can also record observations like $\mathsf{carStarts} = \mathsf{False}$ ('the car did not start') using the $\mathsf{condition} : \mathbb{B} \to 1$ operation (line 4 of fig. 2), or more general 'soft' conditioning using $\mathsf{score} : \mathbb{R}^+ \to 1$. Operationally, every (probabilistic) run of a probabilistic program carries a *weight*. Soft conditioning multiplies the weight, and hard conditioning discards any run where the boolean condition evaluates

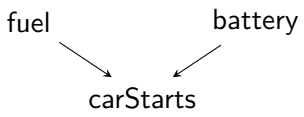


Figure 1: Graphical representaton of the car model

```
fuel    <- bernoulli 0.8
battery <- bernoulli 0.9
carStarts <- fuel && battery
condition (carStarts == False)
return (fuel, battery, carStarts)
```

Figure 2: Probabilistic program version of the car model, conditioned by $\mathsf{carStarts} = \mathsf{False}$

to False, i.e. scores it by 0. Clearly, $\mathsf{condition}(b) = \mathsf{score}(\textbf{if } b \textbf{ then } 1 \textbf{ else } 0)$. [TODO: cite.] The conditional distribution that the program represents is then obtained from a weighted normalisation of the runs.

The problem of inferring the exact conditional distribution from a probabilistic program is undecidable in general [TODO: cite], and intractable to say the least. This is why inference is often solved using approximate algorithms like [TODO: cite].

Semantics of PPLs The semantics of probabilistic programming languages are generally given in a category $\mathbf{C}$ that 'supports probability theory' with a normalised probability monad Prob [TODO: Kock]. In the discrete case, one considers **Set** and the finitely supported distributions monad. Because **Set** is cartesian-closed, one can not only interpret first order programs, but also higher order ones like $(X \to \mathsf{Prob}(Y)) \to \mathsf{Prob}(Z)$. In the more general continuous case, one often uses categories like the category of measurable spaces Meas (or subcategories thereof) with the Giry monad $G : \mathsf{Meas} \to \mathsf{Meas}$ [TODO: cite]. Because Meas is famously not cartesian-closed, several works [37, 16] have developed the category of quasi-Borel spaces (QBS) as an alternative in which higher order probabilistic programming is possible. Many extensions to QBS have since been developed, such as ωQBS [TODO: cite], which enables recursion, and quasi-Borel families (QBFs), which enable dependent types [TODO: cite].

Normalised probability monads only allow one to interpret sampling; interpreting conditioning (or scoring in general) will require *unnormalised* probability monads Dist. An unnormalised probability monad should be a super-monad of the normalised counterpart. Without going into too much detail, there are many possible choices for Dist with different trade-offs. For instance, we can consider $\mathsf{Dist}(X) = \mathsf{Prob}(X \times \mathbb{R}^+)$, which enables general scoring using a writer effect; or indeed the subdistribution monad $\mathsf{Dist}(X) = \mathsf{Prob}(X + 1)$, which only supports (hard) conditioning. Finally, in this context, Bayesian inference can be thought of as a normalisation map $\mathsf{norm} : \mathsf{Dist}(X) \to \mathsf{Prob}(X) + 1$ (e.g. [37]), where the additional $+1$ outcome is because the input distribution could be everywhere-zero.

Many PPLs exploit the monadic interface in their implementation, if not their formal analysis. To name a few:

- MonadBayes [32, 33, 34] encodes the monadic interface using Haskell's type classes, so that programs can be written independently of the choice of inference algorithm.
- LazyPPL [11] exploits Haskell's laziness for both efficient inference and inference on coinductive data like streams.
- GenJAX [4] is a vectorised probabilistic programming language built on top of Jax [TODO: cite], and has its formal analysis based on the monadic language.

Synthetic Foundations In parallel to probabilistic programming, *Markov categories* present a synthetic approach to probability theory. Markov categories axiomatise the categorical structure essential to probability theory without measure theory: a symmetric monoidal category (intuitively, a category of stochastic maps), along with a commutative

comonoid on each object, i.e. a co-multiplication operation $\delta_A : A \to A \otimes A$ and a co-unit $\epsilon_A : A \to I$, such that ϵ_A is natural with respect to A. Intuitively, one thinks of δ_A as the operation of taking an A and copying it, and ϵ_A as the operation of discarding the result of type A. [TODO: citations]. Markov categories provide a unifying account of many categories in which one can do probability theory, notably $\mathsf{FinStoch}$, the category of (finite) stochastic matries; Stoch, the category of measurable spaces and Markov kernels; and the Kleisli categories of the probability monads that we mentioned above. For example, Stoch is the Kleisli category of the Giry monad on Meas. The correspondence betwen Kleisli categories of affine commutative monads and a certain subclass of Markov categories coined 'representable' is studied in [TODO: Fritz, Gonda, Perrone, Rischel 2023].

Markov categories provide an elegant framework to formalise probability theory concepts, notably: deterministic morphisms, almost sure equality, conditionals, stochastic independence, Disintegration, and Bayesian inversion [TODO: citations]. More relevant to us are also analyses of graphical properties used in causal inference like d-seperation [TODO: cite Fritz and Klingler], which we will elaborate on later.

2.2 Foundations of Causal Modelling

Causal inference [25, 26], pioneered by Pearl, is a statistical toolkit that allows one to reason beyond correlation, about causation and counterfactuals. In this section we give a primer of Pearl's framework for Causality, which will serve as motivation to causal probabilistic programming. Part of the section is adapted from the term paper [39].

2.2.1 Pearl's Three Rungs of Causality

Pearl [26] organises causal questions about the world into three rungs:

1. Observational: if we observe that the car didn't start, does it have fuel?
2. Interventional: if we fill the car's fuel tank, will it start?
3. Counterfactual: given that the car didn't start, would it have started had we filled the tank?

As we shall see, Pearl's rungs are mathematically distinct [25], and each of these rungs require a new class of graphical models exposing more intension.

Rung 1: Observation, Bayesian Networks Rung 1 queries are simply Bayesian queries, which can be answered using Bayesian inference given any representation of the joint distribution, like a Bayesian network or a probabilistic program. In our example query, we wish to compute $\Pr(\neg\mathsf{fuel} \mid \neg\mathsf{carStarts})$, which we can simply produce by running inference on fig. 2, or on the corresponding Bayesian network fig. 1. We then obtain $\Pr(\neg\mathsf{fuel} \mid \neg\mathsf{carStarts}) = 5/7 \approx 0.71$.

Rung 2: Intervention, Causal Bayesian Networks Rung 2 queries ask what happens if we *force* a variable to take a value, e.g. if we fill the car's tank. This is a question about causation and cannot be simply answered using the joint distribution.

Pearl's answer is to reinterpret Bayesian networks more intensionally: a *causal Bayesian network* (or *graphical causal model*) is a Bayesian network whose graph is represents a causal structure [25, 36]. That is, an edge $X \to Y$ in a CBN says that X is a direct cause of Y, whereas the same edge in a BN only says that X and Y are correlated. Concretely,

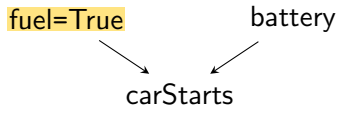
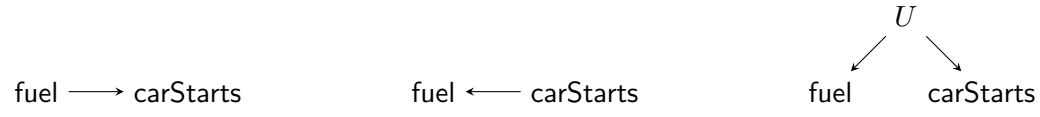


Figure 3: Car model, intervened with $\mathsf{do}(\mathsf{fuel} = \mathsf{True})$



(a) Fuel causes starting (b) Starting causes fuel (c) Common cause

Figure 4: Three causal Bayesian networks over fuel and carStarts equivalent as Bayesian networks.

we can consider CBNs to be BNs with an additional graph-surgery operation called *intervention*: $\mathsf{do}(X = x)$ replaces the mechanism of X by the constant x, cutting X off from its parents and leaving every other mechanism unchanged.

For instance, if we reinterpret fig. 1 to be a CBN, we can answer our example query by first intervening on the model ($\mathsf{do}(\mathsf{fuel} = \mathsf{True})$), producing fig. 3, and then performing inference. We then obtain $\Pr(\mathsf{carStarts} \mid \mathsf{do}(\mathsf{fuel} = \mathsf{True})) = 0.9$.

The distinction between CBNs and BNs lies in the fact that the additional operation of intervention exposes the intensional, causal structure of the graphical model [TODO: cite]. In fig. 4, all three models represent the same joint distribution, so can be seen as equivalent as BNs, or 'rung 1 equivalent'. However, they are different as CBNs, because they clearly behave differently under intervention.

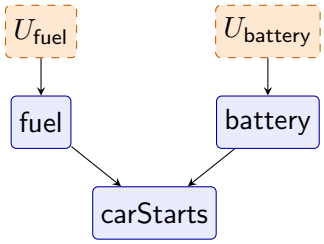
One possible intuition behind interventions is that of scientific experiments, where setting a parameter is akin to performing an intervention. [TODO: cite] So far we only considered *perfect* interventions: forcefully sets a variable at a precise value. Many other types of interventions have since been considered. A few examples include:

•

[TODO: stochastic, soft interventions.]

Rung 3: Counterfactuals, Structural Causal Models At rung 3, we consider queries that reason jointly about multiple 'worlds' at the same time. Our example query *given that the car didn't start, would it have started had we filled the tank?* explicitly reasons about two worlds: a factual world, in which we observe that the car did not in fact start, and a counterfactual world, where we fill the tank using an intervention and ask whether it would have started. Philosophically, we also want the two worlds to be very correlated: only the intervened variable should differ in the two worlds, and everything else should be kept the same. Therefore, we naturally consider defining an operation that takes a causal model and attempts to create two adequately correlated copies of it – what Pearl would call *twinning*. However, unfortunately, the data of a causal Bayesian network (e.g. the model in fig. 1) does not determine a canonical twinning [2]: there are different choices whether to share and how to share randomness between the two copies of the model.

Pearl's answer is to consider an even more intensional class of causal models called *structural causal models* [25] so that twinning is always well-defined. The following definition is adapted from [7].



$$
\begin{aligned}
U_{\text{fuel}} &\sim \text{Bern}(0.8) \\
U_{\text{battery}} &\sim \text{Bern}(0.9) \\
\text{fuel} &:= U_{\text{fuel}} \\
\text{battery} &:= U_{\text{battery}} \\
\text{carStarts} &:= \text{fuel} \wedge \text{battery}
\end{aligned}
$$

Figure 5: The car model as an SCM: graph (left) and structural equations (right). Exogenous variables are dashed (orange), endogenous variables solid (blue).

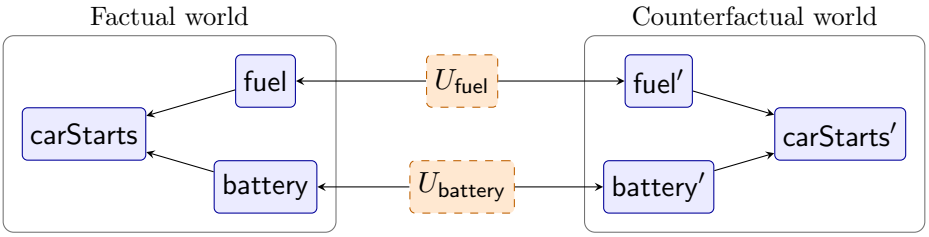


Figure 6: The twinned car SCM. The endogenous variables are copied into a factual and a counterfactual world, while the exogenous variables, and hence all the randomness, are shared between them.

Definition 2.1 (Structural Causal Model)**.** An SCM is a tuple $\mathcal{M} = \langle \mathbf{U}, \mathbf{V}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{f}, \mathbb{P}_{\boldsymbol{X}} \rangle$ where

1. $\mathbf{U}$ is a finite set of exogenous variables.
2. $\mathbf{V}$ is a finite set of endogenous variables, disjoint from $\mathbf{U}$.
3. $\boldsymbol{X}$: for each exogenous variable $u \in \mathbf{U}$, its domain space $X_u \in \mathbf{Ob}$ $(\mathbf{C})$
4. $\boldsymbol{Y}$: for each endogenous variable $v \in \mathbf{V}$, its domain, a measurable space Y_v.
5. $\boldsymbol{f}$: for each $v \in \mathbf{V}$, a tuple $(U_v \subseteq \mathbf{U}, V_v \subseteq \mathbf{V} \backslash \{v\}, f_v : \prod_{u \in U_v} X_u \times \prod_{v' \in V_v} Y_{v'} \to Y_v)$ where f_v is measurable. Intuitively these correspond to structural equations of the form $v = f_v(U_v, V_v)$.
6. $\mathbb{P}_{\boldsymbol{X}}$: a distribution over the exogenous variables.

It is clear that each acyclic SCM is a particular class of CBNs which factor into an exogenous part that contains all the randomness (intuitively what is shared across the worlds), and an endogenous part that is fully deterministic and contains the causal structure (what is copied to different worlds). Thanks to this factorisation, twinning can then be defined as the operation of creating 2 (or, in general, n) copies of the endogenous part and sharing the exogenous part. For example, one might define a car model SCM as fig. 5, and its twinned version becomes fig. 6. Once we have the twinned model, we can then treat it as a CBN and perform the $\mathsf{do}(\mathsf{fuel}' = \mathsf{True})$ intervention on the counterfactual copy, and condition by $\mathsf{carStarts} = \mathsf{False}$ on the factual copy, to compute the result to our query: $\Pr(\mathsf{carStarts}' \mid \neg\mathsf{carStarts}, \mathsf{do}(\mathsf{fuel}' = \mathsf{True})) = \frac{9}{14} \approx 0.64$.

[TODO: finish extra intension explanation: CBNs equivalent but diff as SCMs] The extra intension is *how the randomness is generated*. Every SCM induces a causal Bayesian network by pushing the noise through the structural equations, but different SCMs can

induce the same one. Take $X := U_X$ and $U_X, U_Y \sim \text{Bern}(1/2)$ in two models, with $Y := U_Y$ in the first and $Y := X \oplus U_Y$ in the second. In both, Y is a fair coin under every intervention on X, so they are equivalent as causal Bayesian networks. They are not equivalent as SCMs: having observed $X = Y = 1$, had X been 0, Y would still have been 1 in the first model, but 0 in the second.

2.2.2 Semantics of Causality

Causal Models

Causal Structures

2.3 Causal Probabilistic Programming Languages

2.3.1 Summary of our POPL 2027 paper

Motivation: Causal models as probabilistic programs, operations as program transformations

Contribution 1: semantic foundations of causal programming Our first contribution is to develop a semantic foundation of causal models by taking the analogy with probabilistic programs seriously. In **??**, we reconstruct Pearl's rungs of causality as a hierarchy of simple first order probabilistic programming languages L_1, L_2, L_3. Each language exposes more intension than the previous one using syntactic annotations, and causal inference is performed using program transformations that consume these syntactic annotations: L_1 is a standard first order PPL; L_2 exposes typed holes named *intervention labels*, canonically of type Maybe(A), allowing substituting to optionally perform an intervention; L_3 annotates each let binding with **exo** or **endo**, so that twinning can be formulated as a vectorisation transform. The same idea underlies the intervention-as-program-transformation view of Witty [41, 43], the logical account by Ibeling and Icard [17], and the string-diagrammatic accounts of Jacobs et al. [19] and Lorenz and Tull [23].

What is new is the observation that this simple setting yields a semantic characterisation of Pearl's rungs of causality (**??**). We develop a unified account of contextual equivalences that recovers the full spectrum of causal model equivalences by varying the class of admissible contexts, e.g. Rung 3 (L_3) programs are Rung 3 equivalent ($\equiv_3$) if they behave the same under all twinnings and interventions; Rung 2 equivalent ($\equiv_3^{\mathsf{intv}}$) if under all interventions; Rung 1 equivalent ($\equiv_3^{\mathsf{obs}}$) if they induce the same distribution under no intervention. The subscript 3 marks that we compare L_3 programs throughout; the superscript marks the rung of context tested against (obs, intv, or none for full rung-3 twinning). These equivalences form a strict inclusion hierarchy $\equiv_3 \subsetneq \equiv_3^{\mathsf{intv}} \subsetneq \equiv_3^{\mathsf{obs}}$, an analogue of the causal hierarchy's non-collapse [3], i.e. programs indistinguishable at a lower rung may still differ at a higher one.

In **??** we connect the framework to the theory of SCMs, showing that L_3 extends SCMs conservatively: each SCM can be compiled into L_3, preserving the semantics of intervention and twinning (**??**). Through this compilation, we show that the hierarchy of SCM equivalences developed by Bongers et al. [7] is naturally subsumed by our program equivalences (**??**).

Contribution 2: Typed abstractions and denotational semantics for expressive causal probabilistic programming Our second contribution is the development of L_{HO}, a semantically-founded, expressive, and typed language for higher-order causal probabilistic programming. L_{HO} has a similar programming model to ChiRho [10, 40]: it is expressive as a programming language (higher order, with control flow) and as a causal calculus (rich, higher order interventions, see **??**); we illustrate it with examples such as mediation analysis, which decomposes a causal effect into a direct path and a path through an intermediate variable (**??**). However, unlike ChiRho, it extends the first-order languages in a semantic-preserving way (**??**).

Rung 2: To manage interventions, we track intervention labels using *grading* [24, 20, 12, 13, 31], similarly to the **ImP** construction in [22] or intervention points in [40]. We define a graded reader monad $\mathcal{I}_{[\tau_1 \ldots \tau_n]}(X) \triangleq \prod_i \tau_i \to \mathsf{Meas}(X)$ graded by lists of intervention types $[\tau_1 ... \tau_n]$, where Meas is the unnormalised probability monad. A causal model of type $\mathcal{I}_{[\tau_1 \ldots \tau_n]}(X)$ is an intervenable probabilistic program with n typed intervenable holes, each of type τ_i, and a do intervention on the ith hole is simply partial application.

Rung 3: To manage counterfactuals, we first note that we will consider *n-world twinning*, which creates n copies of the original model, because general counterfactual reasoning may require more than 2 copies. We now ask: what is the type of a model under twinning? The twinned version of the car model where only `fuel` is intervenable and `carStarts` is returned has type $\mathcal{I}_{[(\mathrm{Maybe}(\mathbb{B}))^n]}(\mathbb{B}^n)$: an intervenable computation with n intervention labels (one `fuel` intervention label per world), and returning n copies of `carStarts`. The trick is to make each causal model carry its behaviour under twinning, for any world count n. In other words the *twinnable* car model is a world-polymorphic vectorised computation of type $\forall n. \mathcal{I}_{[(\mathrm{Maybe}(\mathbb{B}))^n]}(\mathbb{B}^n)$, where $\forall$ denotes System-F-style polymorphism [30, 15]. Endogenous variables like `carStarts` become n-dimensional vectors, where n is the number of worlds, and compose together using the applicative interface [14]. Twinning is then a simple type application. The corresponding technical setting is the functor category $[\mathbf{Bij}, \mathbf{C}]$, where $\mathbf{C}$ is the semantic model of the underlying PPL, e.g. $\mathbf{C} = \mathbf{Qbs}$ if it is continuous and higher order. Each type A is informally an n-indexed family of $\mathbf{C}$-objects $[\![A]\!](n)$, and each function $A \to B$ is interpreted as an n-indexed family of morphisms $[\![A]\!](n) \to [\![B]\!](n)$, i.e. its behaviour under n-world twinning. This use is similar to names in nominal sets [28].

We derive L_{HO} by denotational and functional programming techniques. In **??**, we develop a denotational semantics for our hierarchy of languages, which we show equivalent to the operational description (**????**). We then turn the categorical abstractions we develop into functional programming abstractions (**??**).

2.3.2 Prior existing causal languages and libraries

Many practical tools like DoWhy (part of the PyWhy ecosystem [29]) are libraries for manipulating SCMs as graphs and thus have limited expressiveness. DoWhy's GCM module nonetheless reaches rung 3, computing sample-level counterfactuals on fitted mechanisms [6]. Among causal PPLs proper, several stop at rung 2. Brulé's Whittemore [8], an early embedded causal DSL, covers rung-2 identification and estimation. Pyro provides a do effect handler [5], the design that ChiRho generalises; PyMC's do performs graph surgery on the computation graph [1, 18]. Witty et al. [42] treat an intervention as a source transformation that replaces subterms, powerful (shift, variance-scaling, mechanism replacement) but non-compositional. In contrast, our languages only expose simple typed holes, which generalise to the same powerful classes of interventions for free at higher-order (**??**). Rung-3 queries can sometimes be re-encoded on such systems by hand

by an identifiability argument [35].

Four systems support rung 3 natively [38, 10, 27, 21]. **OmegaC** [38] obtains rung 2 through a non-standard lazy evaluation strategy: a let-binding evaluates the bound expression e to a closure $\langle \Gamma, e \rangle$ and $\mathsf{do}(x \mapsto e')$ retroactively modifies that closure $\langle \Gamma[x \mapsto e'], e \rangle$, reminiscent of dynamic binding. Rung 3 is enabled by making each program by construction an SCM-like deterministic function of an explicit exogenous noise ω. OmegaC is limited in expressiveness (sampling and building random variables are separate), and only has an operational semantics. We conjecture our formalism can model OmegaC's core denotationally. **MultiVerse** [27] realises Pearl's abduction–action–prediction recipe as an importance-sampling engine, without a formal semantics. **ChiRho** [10, 9], built from effect handlers and tensor broadcasting, is both the system closest to ours, and our initial motivation. Its recent distillation into typed functional abstractions [40] bears clear similarities to L_{HO}: intervention *points* tracked in a reader monad (our *labels* leave open how an intervention is consumed), vectorisation used in an applicative way for twinning. Two differences are key. First, our rungs 2 and 3 are decoupled: interventions never create worlds; twinning alone does. Under ChiRho's multi-world handler, by contrast, each intervention doubles the number of worlds. Second, L_{HO} fixes the number of worlds at twinning time and whether a variable is vectorised (X or $\mathcal{E}(X)$) determines whether it is exogenous or endogenous. In contrast, ChiRho vectorises its values by default (MVal in the typed distillation [40]) and encodes an implicit sharing inference using tensor broadcasting: a random draw is shared wherever broadcasting permits, while sample sites downstream of an intervention will draw fresh noise per world. ChiRho's MVal thus allows finer control over sharing than L_{HO}, but its relation to Pearl's twinning semantics is less clear. Finally, **probabilistic logic programming** also reaches rung 3: Kiesel et al. [21] answer counterfactual queries on ProbLog programs by a transformation proved correct against Pearl's semantics.

2.4 Brief overview of other notions of causality

2.4.1 Logical, Diagrammatic, Categorical notions of Causality

Functorial Semantics

Logical characterisation of Ibeling and Icard

Modulating Normalisation

2.4.2 "Causality" in other fields

Concurrency

Quantum Computing

3 Project Proposal

3.1 Summary of the current work done

3.2 Developing the field of Causal PPLs

3.2.1 Putting the language into practice

3.2.2 Expressivity

3.3 Better understanding our categorical constructions

3.3.1 Relation to other constructions

3.3.2 A more synthetic approach?

3.4 Understanding causality using a graph-centered approach

3.5 Understanding relationship with 'causality' as used in other fields

3.5.1 Concurrency

3.5.2 Higher-order processes and quantum causality

References

[1] Oriol Abril-Pla et al. "PyMC: a modern, and comprehensive probabilistic programming framework in Python". In: *PeerJ Computer Science* 9 (2023), e1516. DOI: 10.7717/peerj-cs.1516.

[2] Alexander Balke and Judea Pearl. "Probabilistic evaluation of counterfactual queries". In: *Proceedings of the Twelfth AAAI National Conference on Artificial Intelligence*. AAAI'94. Seattle, Washington: AAAI Press, 1994, pp. 230–237.

[3] Elias Bareinboim et al. "On Pearl's Hierarchy and the Foundations of Causal Inference". In: *Probabilistic and Causal Inference: The Works of Judea Pearl*. 1st. New York, NY, USA: Association for Computing Machinery, 2022, pp. 507–556.

[4] McCoy R. Becker et al. "Probabilistic Programming with Vectorized Programmable Inference". In: *Proc. ACM Program. Lang.* 10.POPL (Jan. 2026). DOI: 10.1145/3776729. URL: https://doi.org/10.1145/3776729.

[5] Eli Bingham et al. "Pyro: Deep Universal Probabilistic Programming". In: *Journal of Machine Learning Research* (2018).

[6] Patrick Blöbaum et al. "DoWhy-GCM: An extension of DoWhy for causal inference in graphical causal models". In: *Journal of Machine Learning Research* 25 (2024). URL: https://jmlr.org/papers/v25/22-1258.html.

[7] Stephan Bongers et al. "Foundations of structural causal models with cycles and latent variables". In: *The Annals of Statistics* 49.5 (Oct. 2021). ISSN: 0090-5364. DOI: 10.1214/21-aos2064. URL: http://dx.doi.org/10.1214/21-AOS2064.

[8] Joshua Brulé. *Whittemore: An embedded domain specific language for causal programming*. 2018. arXiv: 1812.11918 [cs.AI]. URL: https://arxiv.org/abs/1812.11918.

[9] *Causal probabilistic programming without tears - ChiRho documentation*. https://basisresearch.github.io/chirho/tutorial_i.html. Accessed: 2025-10-30.

[10] *ChiRho*. https://basisresearch.github.io/chirho. Accessed: 2025-10-30.

[11] Swaraj Dash et al. “Affine Monads and Lazy Structures for Bayesian Programming”. In: *Proc. ACM Program. Lang.* 7.POPL (Jan. 2023). DOI: 10.1145/3571239. URL: https://doi.org/10.1145/3571239.

[12] Ulrich Dorsch, Stefan Milius, and Lutz Schröder. “Graded Monads and Graded Logics for the Linear Time–Branching Time Spectrum”. In: *CONCUR 2019* (2019).

[13] Soichiro Fujii, Shin-ya Katsumata, and Paul-André Melliès. “Towards a formal theory of graded monads”. In: *International Conference on Foundations of Software Science and Computation Structures*. Springer. 2016, pp. 513–530.

[14] Jeremy Gibbons. “APLicative programming with Naperian functors (extended abstract)”. In: *Proceedings of the 1st International Workshop on Type-Driven Development*. TyDe 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 13–14. ISBN: 9781450344357. DOI: 10.1145/2976022.2976023. URL: https://doi.org/10.1145/2976022.2976023.

[15] Jean-Yves Girard. “Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur”. PhD thesis. Université Paris 7, 1972.

[16] Chris Heunen et al. “A convenient category for higher-order probability theory”. In: *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, June 2017, pp. 1–12. DOI: 10.1109/lics.2017.8005137. URL: http://dx.doi.org/10.1109/LICS.2017.8005137.

[17] Duligur Ibeling and Thomas Icard. “Probabilistic Reasoning across the Causal Hierarchy”. In: *CoRR* abs/2001.02889 (2020). arXiv: 2001.02889. URL: http://arxiv.org/abs/2001.02889.

[18] *Interventional distributions and graph mutation with the do-operator — PyMC example gallery*. https://www.pymc.io/projects/examples/en/latest/causal_inference/interventional_distribution.html. Accessed: 2026-07-05.

[19] Bart Jacobs, Aleks Kissinger, and Fabio Zanasi. “Causal inference by string diagram surgery”. In: *Foundations of Software Science and Computation Structures: 22nd International Conference, FOSSACS 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6–11, 2019, Proceedings 22*. Springer. 2019, pp. 313–329.

[20] Shin-ya Katsumata. “Parametric effect monads and semantics of effect systems”. In: *SIGPLAN Not.* 49.1 (Jan. 2014), pp. 633–645. ISSN: 0362-1340. DOI: 10.1145/2578855.2535846. URL: https://doi.org/10.1145/2578855.2535846.

[21] Rafael Kiesel, Kilian Rückschloß, and Felix Weitkämper. ““What if?” in probabilistic logic programming”. In: *Theory and Practice of Logic Programming* 23.4 (2023).

[22] Jack Liell-Cock and Sam Staton. “Compositional imprecise probability: A solution from graded monads and markov categories”. In: *Proceedings of the ACM on Programming Languages* 9.POPL (2025), pp. 1596–1626.

[23] Robin Lorenz and Sean Tull. *Causal models in string diagrams*. 2023. arXiv: 2304.07638 [cs.LO]. URL: https://arxiv.org/abs/2304.07638.

[24] Dominic Orchard, Philip Wadler, and Harley Eades III. “Unifying graded and parameterised monads”. In: *Proc. MSFP 2020* (2020).

[25] Judea Pearl. *Causality*. Cambridge University Press, 2009.

[26] Judea Pearl and Dana Mackenzie. *The Book of Why: The New Science of Cause and Effect*. 1st. USA: Basic Books, Inc., 2018. ISBN: 046509760X.

[27] Yura Perov et al. “Multiverse: causal reasoning using importance sampling in probabilistic programming”. In: *Symposium on advances in approximate bayesian inference*. PMLR. 2020, pp. 1–36.

[28] Andrew M. Pitts. *Nominal Sets: Names and Symmetry in Computer Science*. USA: Cambridge University Press, 2013. ISBN: 1107017785.

[29] *PyWhy, An Open Source Ecosystem for Causal Machine Learning*. https://www.pywhy.org/. Accessed: 2025-10-30.

[30] John C. Reynolds. “Towards a theory of type structure”. In: *Programming Symposium, Proceedings Colloque Sur La Programmation*. Berlin, Heidelberg: Springer-Verlag, 1974, pp. 408–423. ISBN: 3540068597.

[31] Ralph Sarkis and Fabio Zanasi. *Graded String Diagrams for Imprecise Probability and Causal Intervention*. 2026. arXiv: 2501.18404 [math.CT]. URL: https://arxiv.org/abs/2501.18404.

[32] Adam Ścibior, Zoubin Ghahramani, and Andrew D Gordon. “Practical probabilistic programming with monads”. In: *Proceedings of the 2015 ACM SIGPLAN Symposium on Haskell*. 2015, pp. 165–176.

[33] Adam Ścibior, Ohad Kammar, and Zoubin Ghahramani. “Functional programming for modular Bayesian inference”. In: *Proceedings of the ACM on Programming Languages* 2.ICFP (2018), pp. 1–29.

[34] Adam Michał Ścibior. “Formally justified and modular Bayesian inference for probabilistic programs”. PhD thesis. 2019.

[35] Ilya Shpitser and Judea Pearl. “What counterfactuals can be tested”. In: *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence*. UAI’07. Vancouver, BC, Canada: AUAI Press, 2007, pp. 352–359. ISBN: 0974903930.

[36] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. The MIT Press, Jan. 2001. ISBN: 9780262284158. DOI: 10.7551/mitpress/1754.001.0001. URL: https://doi.org/10.7551/mitpress/1754.001.0001.

[37] Sam Staton et al. “Semantics for probabilistic programming: higher-order functions, continuous distributions, and soft constraints”. In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*. LICS ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 525–534. ISBN: 9781450343916. DOI: 10.1145/2933575.2935313. URL: https://doi.org/10.1145/2933575.2935313.

[38] Zenna Tavares et al. “A language for counterfactual generative models”. In: *International conference on machine learning*. PMLR. 2021, pp. 10173–10182.

[39] Théo Wang et al. “From structural causal models to higher order causal probabilistic programming: contextual equivalences, graded monads and functor categories”. In: *Proceedings of the 54th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2027)*. To appear. 2027.

[40] Théo Wang et al. *Typed Abstractions for Causal Probabilistic Programming*. Extended abstract presented at LAFI 2026: Workshop on Languages for Inference, co-located with POPL 2026. Non-archival workshop contribution. Rennes, France, Jan. 2026. URL: https://www.cl.cam.ac.uk/~jdy22/papers/typed-abstractions-for-causal-probabilistic-programming.pdf.

[41] Sam Witty, David Jensen, and Vikash Mansinghka. "SBI: A Simulation-Based Test of Identifiability for Bayesian Causal Inference". In: *arXiv preprint arXiv:2102.11761* (2021).

[42] Sam Witty et al. "Bayesian causal inference via probabilistic program synthesis". In: *CoRR* abs/1910.14124 (2019). arXiv: 1910.14124. URL: http://arxiv.org/abs/1910.14124.

[43] Sam A Witty. "Bayesian Structural Causal Inference with Probabilistic Programming". PhD thesis. University of Massachusetts Amherst, 2023.